University of Camerino

School of Science and Technologies

Master Degree in Computer Science (Class LM-18)

Course of Knowledge Engineering

Credit card fraud detection system

Group members
Nicola Lerco (Student ID 135770)
Riccardo Peruzzi (Student ID 135786)

Supervisor
Prof. Dr. Knut Hinkelmann

A.A. 2025/2026

Contents

List of Figures

List of Tables

1. Introduction

1.1 Project description

This project concerns the development of an intelligent system for detecting fraud in credit card transactions, based on the application of knowledge representation and reasoning principles. The main objective is to implement a decision model that operates according to a risk-based approach, calculating a risk score for each individual financial operation and triggering appropriate responses when critical thresholds are exceeded. The system analyzes several variables associated with each transaction: the payment amount, the geographical location of both the cardholder and the merchant, and the type of service used (physical or online store). Based on these inputs, predefined business rules assign partial risk scores that are aggregated into a final score, which determines whether a transaction should be approved, suspended for human review, or automatically rejected. The project is developed incrementally across multiple tasks, each introducing new requirements and formalisms. The decision logic is first modeled using the DMN (Decision Model and Notation) standard, a specification developed by the Object Management Group (OMG) that provides a structured and interoperable way to represent repeatable business decisions. The knowledge base is then extended and reimplemented using Prolog, a logic programming language well suited for rule-based reasoning and relational queries over structured data. In the final stage, the domain knowledge is further formalized as a Knowledge Graph using Protégé: an RDF schema is defined to capture the relevant concepts and relationships, the test cases are represented as RDF instances, and SPARQL queries are used to retrieve transactions of interest. SWRL rules are additionally introduced to derive, through automated inference, whether each transaction should be classified as fraudulent.

2. Requirements

2.1 First Task

The decision model for fraud detection must satisfy the following functional and structural requirements.

Inputs

The system requires three input variables for each transaction: the merchant's country (`Country_Merchant`), the cardholder's country (`Country_Carholder`), and the transaction amount (`Amount`).

Sub-decisions

The model must be structured into modular sub-decisions. A geographic mapping decision (`Country_Continent`) must convert raw country names into continent-level categories (specifically Italy, Europe, Africa, Switzerland, and Other) to allow continent-based risk rules to be applied uniformly. Four independent risk-scoring sub-decisions must then compute partial scores: `Merchant_Risk` based on where the card is used, `Carholder_Risk` based on where the cardholder resides, `Special_Risk` for the specific case of a Swiss cardholder transacting in South Africa, and `Amount_Risk` based on the transaction value.

Main decision

A `Total_Risk` decision must aggregate all partial scores into a single numerical risk score. A final `Result` decision must then evaluate the total score and produce one of three outcomes: approved (score ≤ 100), stopped for human review (score > 100), or rejected (score > 150).

Scoring rules

The risk increments must reflect the business rules defined in the project specification, as summarized in Table 2.1.

Test scenarios

The model must be validated against three predefined scenarios:

- **Mr. Mensoni** from Italy, purchasing in South Africa for €11,234.

Category	Condition	Risk Points
Merchant location	Italy	+10
	Europe	+20
	Africa	+60
Cardholder location	Italy	+0
	Other European country	+10
	Africa	+20
Special case	Swiss cardholder in South Africa	+30
Amount	Less than €1,000	+10
	Up to €10,000	+60
	Above €10,000	+90

Table 2.1: Risk scoring rules applied per transaction.

- **Mr. Meyer** from Germany, paying in Morocco for €12.
- **Mr. Suter** from Switzerland, paying in South Africa for €90.

2.2 Second Task

The second task extends the fraud detection system with an additional behavioral rule based on temporal and geographical analysis across multiple transactions, and requires a full reimplementation of the decision logic as a Prolog knowledge base.

Additional fraud rule

The system must detect the case in which the same cardholder's credit card is used in two different physical stores located in two different countries within a time window of 5 minutes (300 seconds). When this condition is met, the risk score for the involved transactions must be raised to at least 150, regardless of the score computed by the standard rules.

DMN extension

The applicability of the DMN standard to this new rule must be discussed. If an extension of the model defined in Task 1 is feasible, it must be designed and submitted; otherwise, a motivated explanation of why the rule cannot be expressed within the DMN formalism must be provided.

Ontology

An ontology must be defined to capture the core terminology of the domain. Concepts such as transaction, cardholder, and merchant must be represented as classes, their relationships as object properties, and the specific instances used in the test scenarios as individuals.

Prolog knowledge base

A knowledge base must be implemented in Prolog that incorporates all fraud detection rules defined up to this point, including both the scoring rules from Task 1 and the new temporal comparison rule. The implementation must represent transactions, cardholders, and merchants as structured facts, and expose appropriate queries to detect and classify fraudulent transactions.

Test scenarios

The additional rule must be validated against the following scenario:

- **Mr. Mensoni**'s card is used at the restaurant "Winds of the Deserts" in Morocco for 500 CHF, only 3 minutes after the purchase of the wooden mask in South Africa — corresponding to transactions `tx1` and `tx4` in the knowledge base.

2.3 Third task

This task requires converting the developed solution to Prolog within a knowledge graph, using the Protégé ontology editor. The goal is to define an RDF schema that models the domain, represent test cases as individuals in the ontology, write SPARQL queries to extract relevant information, and implement SWRL rules that automatically derive the decision on each transaction based on previously defined risk criteria.

RDF Schema

For this step, it is necessary to develop an ontology in Protégé that defines the fundamental classes of the domain. The object properties to relate these classes and the data properties to store concrete values must also be specified.

Test case in RDF

Once the schema has been defined, the individuals corresponding to the scenarios provided by the text must be inserted into the ontology. Two additional transactions should also be added to verify the correct functioning of the five-minute time rule.

SPARQL query

This section should contain SPARQL queries. The first retrieves a list of all transactions in the knowledge graph. The second selects transactions whose trader is based in South Africa. The third returns transactions whose cardholders reside in a European country. Finally, the fourth extracts transactions with Italian merchants and shows the corresponding risk associated with the merchant for each.

SWRL rule

The risk rules defined in the previous tasks should be translated into SWRL rules. Specifically, the rules for the geographic risk of the merchant and cardholder, for the amount-based risk, for the special risk involving Swiss cardholders and South African merchants, and finally the time rule that detects two physical transactions in the same

cardholder in different countries within five minutes should be implemented. A sum rule combines all contributions, while three separate rules assign the final decision based on the 100- and 150-point thresholds.

SPARQL for fraudulent transactions

After activating the Protégé reasoner to enforce SWRL rules, a SPARQL query should be written that retrieves only transactions considered fraudulent.

3. Task 1

3.1 Decision Model

Decision Model and Notation (DMN) is a standard developed by the Object Management Group (OMG) and widely used in business analysis. It provides a structured way to represent and model repeatable decisions within organizations, ensuring that decision logic can be shared and reused across different systems and companies. By using DMN, organizations can design clear and consistent decision models that support decision-making processes and the management of business rules.

3.2 DRD overview

The DRD (decision requirements diagram) model is structured as a pipeline of sub-decisions that independently compute partial risk scores, which are then aggregated into a final risk assessment. The three input variables — `Amount`, `Country_Merchant`, and `Country_Carholder` — feed into dedicated decision nodes, whose outputs converge into the `Total_Risk` decision. A final `Result` decision then evaluates the total score against predefined thresholds to determine the outcome of the transaction.

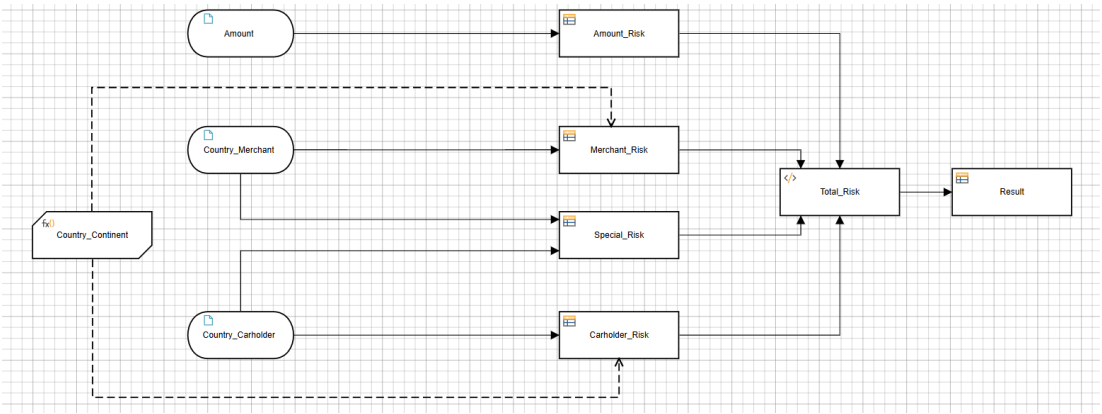

Figure 3.1: Overall structure of the DMN fraud detection model.

As shown in Figure 3.1, the model makes use of a Business Knowledge Model (BKM) called `Country_Continent`, which is invoked by the risk-scoring decisions to map raw country names into continent-level categories.

3.3 Input variables

The model accepts three input variables per transaction. The first is `Amount`, a numerical value representing the transaction amount in euros.

The other two inputs, `Country_Merchant` and `Country_Carholder`, share a custom enumerated type called `Country`, defined to restrict the accepted values to a predefined list of valid nations. This type encompasses all European and African countries relevant to the model, as listed in Table 3.1.

Region	Countries
Europe	Albania, Andorra, Austria, Belarus, Belgium, Bosnia and Herzegovina, Bulgaria, Croatia, Cyprus, Czech Republic, Denmark, Estonia, Finland, France, Germany, Greece, Hungary, Iceland, Ireland, Italy, Kosovo, Latvia, Liechtenstein, Lithuania, Luxembourg, Malta, Moldova, Monaco, Montenegro, Netherlands, North Macedonia, Norway, Poland, Portugal, Romania, San Marino, Serbia, Slovakia, Slovenia, Spain, Sweden, Switzerland, Ukraine, United Kingdom, Vatican City
Africa	Algeria, Angola, Benin, Botswana, Burkina Faso, Burundi, Cabo Verde, Cameroon, Central African Republic, Chad, Comoros, Democratic Republic of the Congo, Djibouti, Egypt, Equatorial Guinea, Eritrea, Eswatini, Ethiopia, Gabon, Gambia, Ghana, Guinea, Guinea-Bissau, Ivory Coast (Côte d'Ivoire), Kenya, Lesotho, Liberia, Libya, Madagascar, Malawi, Mali, Mauritania, Mauritius, Morocco, Mozambique, Namibia, Niger, Nigeria, Republic of the Congo, Rwanda, Sao Tome and Principe, Senegal, Seychelles, Sierra Leone, Somalia, South Africa, South Sudan, Sudan, Tanzania, Togo, Tunisia, Uganda, Zambia, Zimbabwe

Table 3.1: Countries included in the custom `Country` type.

3.4 Business Knowledge Model: Country_Continent

The `Country_Continent` Business Knowledge Model (BKM) acts as a reusable function invoked by the risk-scoring decisions. It takes a single input of type `Country` and returns a continent-level category, mapping each country to one of the two following continents: `"Europe"`, `"Africa"`.

The mapping logic is defined using hit policy **F** (First), meaning the first matching rule is applied. As shown in Table 3.2, specific countries are matched explicitly in the first five rules, while rule 6 acts as a catch-all fallback returning `"Other"` for any country not otherwise matched.

For space reasons, not all country-to-continent mappings are listed; the `...` rows represent the omitted entries.

3.5 Sub-decisions

3.5.1 Amount_Risk

The `Amount_Risk` decision computes the risk contribution of the transaction amount. It takes `Amount` as a numerical input and returns a partial risk score based on three

Rule	Country_Merchant	Country_Continent
1	`"Italy"`	`"Europe"`
2	`"Switzerland"`	`"Europe"`
3	`"Germany"`	`"Europe"`
4	`"Morocco"`	`"Africa"`
5	`...`	
6	`"South Africa"`	`"Africa"`
7	`...`	

Table 3.2: Decision table of the `Country_Continent` BKM (hit policy: F).

mutually exclusive ranges. It uses hit policy **U** (Unique), since each possible amount falls into exactly one interval.

Rule	Amount	Amount_Risk
1	< 1000	10
2	[1000..10000]	60
3	> 10000	90

Table 3.3: Decision table of `Amount_Risk` (hit policy: U).

Amounts below €1,000 contribute 10 points, amounts between €1,000 and €10,000 contribute 60 points, and amounts above €10,000 contribute 90 points.

3.5.2 Merchant_Risk

The `Merchant_Risk` decision evaluates the geographical risk associated with the merchant's location. It invokes the `Country_Continent` BKM on the `Country_Merchant` input to determine the continent category and returns the corresponding risk score. A **Unique (U)** hit policy is applied, ensuring that the rules are mutually exclusive and only one rule triggers for any given input. To maintain completeness, a specific rule is defined to return a risk score of 0 for cases that do not match the primary geographical criteria.

Rule	Country_Continent	Country_Merchant	Merchant_Risk
1	`"Europe"`	`"Italy"`	10
2	`"Europe"`	`not ("Italy")`	20
3	`"Africa"`	—	60
4	`"Other"`	—	0

Table 3.4: Decision table of `Merchant_Risk` (hit policy: U).

Transactions in Italy contribute 10 points, in Europe (excluding Italy) 20 points, and in Africa 60 points.

3.5.3 Carholder_Risk

The `Carholder_Risk` decision evaluates the geographical risk associated with the residence of the cardholder. It invokes the `Country_Continent` BKM on the `Country_Carholder` input and returns a partial risk score. Hit policy **U** (Unique) is applied.

Rule	Country_Continent	Country_Cardholder	Cardholder_Risk
1	"Europe"	"Italy"	0
2	"Europe"	not ("Italy")	10
3	"Africa"	—	20
4	"Other"	—	0

Table 3.5: Decision table of `Merchant_Risk` (hit policy: U).

Italian cardholders contribute 0 points, European cardholders 10 points, and African cardholders 20 points. Any other cardholder location also returns 0.

3.5.4 Special_Risk

The `Special_Risk` decision captures a specific business rule that cannot be expressed through continent-level aggregation alone: a Swiss cardholder using their card in South Africa immediately adds 30 points to the risk score. It takes both `Country_Merchant` and `Country_Carholder` as direct inputs, without invoking the BKM. Hit policy **F** (First) is used, with a catch-all rule returning 0 for all other combinations.

Rule	Country_Merchant	Country_Carholder	Special_Risk
1	"South Africa"	"Switzerland"	30
2	—	—	0

Table 3.6: Decision table of `Special_Risk` (hit policy: F).

3.5.5 Main decision: Total_Risk

The `Total_Risk` decision does not use a decision table. Instead, it is implemented as a literal expression that aggregates the four partial risk scores into a single numerical value:

```
Amount_Risk + Merchant_Risk + Carholder_Risk + Special_Risk
```

This design keeps the aggregation logic minimal and transparent, delegating all rule-based reasoning to the upstream sub-decisions.

3.6 Final decision: Result

The `Result` decision evaluates the aggregated `Total_Risk` score and produces a final outcome for the transaction. It uses hit policy **U** (Unique), since the three score ranges are mutually exclusive and exhaustive.

Rule	Total_Risk	Result
1	≤ 100	"Accepted"
2	(100..150]	"Blocked"
3	> 150	"Rejected"

Table 3.7: Decision table of `Result` (hit policy: U).

A transaction is accepted when the total risk score does not exceed 100 points. Scores between 101 and 150 points trigger a block, suspending the transaction for human review. Scores above 150 points result in an automatic rejection.

3.7 Test scenarios and validation

The model was validated against three predefined scenarios. For each case, the partial risk scores produces by the sub-decision are summed into a finel `Total_Risk` value, which is the evaluated by the `Result decision`.

Scenario 1: Mr. Mensoni

Mr. Mensoni is an Italian cardholder purchasing a wooden mask at a physical store in South Africa for €11,234.

Sub-decision	**Score**
`Merchant_Risk` (South Africa → Africa)	60
`Carholder_Risk` (Italy)	0
`Special_Risk` (Italy + South Africa)	0
`Amount_Risk` (€11,234 > €10,000)	90
Total_Risk	**150**

Table 3.8: Risk score breakdown for Mr. Mensoni.

The total score of 150 falls within the range (100..150], resulting in a `Blocked` outcome.

Scenario 2: Mr. Meyer

Mr. Meyer is a German cardholder paying for a meal in Morocco for €12.

Sub-decision	**Score**
`Merchant_Risk` (Morocco → Africa)	60
`Carholder_Risk` (Germany → Europe)	10
`Special_Risk` (Germany + Morocco)	0
`Amount_Risk` (€12 < €1,000)	10
Total_Risk	**80**

Table 3.9: Risk score breakdown for Mr. Meyer.

The total score of 80 does not exceed 100, resulting in an `Accepted` outcome.

Scenario 3: Mr. Suter

Mr. Suter is a Swiss cardholder paying for a meal in South Africa for €90.

The totat score of 100 satisfies the condition ≤ 100, resultin in a `Accepted` outcome. Note that the `SCpecial_Risk` rule contributes 30 additional points due to the specific combination of a Swiss cardholder transacting in South Africa, whic would otherwise have gone undetected by coninent-level rules alone.

Sub-decision	**Score**
`Merchant_Risk` (South Africa → Africa)	60
`Carholder_Risk` (Switzerland → Other)	0
`Special_Risk` (Switzerland + South Africa)	30
`Amount_Risk` (€90 < €1,000)	10
Total_Risk	**100**

Table 3.10: Risk score breakdown for Mr. Suter.

4. Task 2

4.1 Addition fraud rule

The additional "5-minute rule" cannot be fully implemented using only DMN because DMN is mainly designed to evaluate a single transaction independently from other events. In this scenario, however, the decision depends on comparing multiple transactions performed by the same cardholder over time. More specifically, the system must verify whether two transactions occur in different countries within a time interval of less than five minutes. This requires access to historical transaction data and temporal reasoning capabilities, which are outside the scope of standard DMN decision tables. For this reason, the rule should be implemented through an external mechanism, such as a rule engine or a logic-based system.

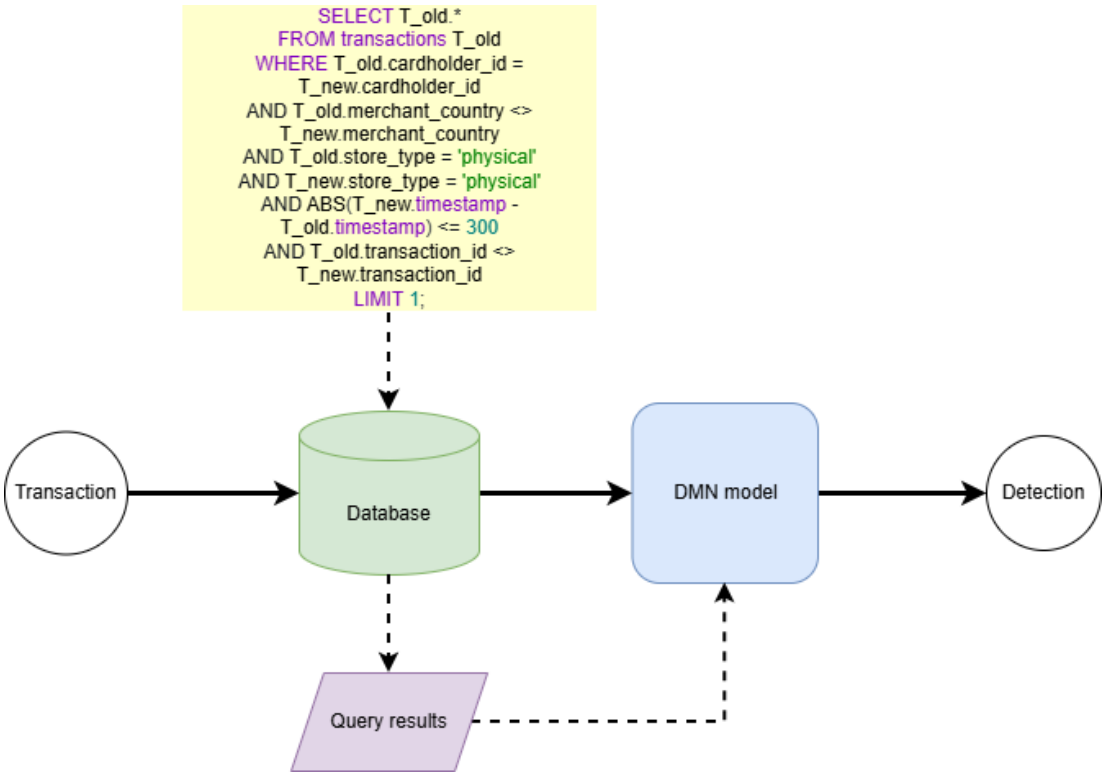

Figure 4.1: Possible solution architecture

A possible solution to overcome this limitation is to integrate the DMN model with an external database through a query-based pipeline. When a new transaction is received, before being passed to the DMN engine, a query is executed on the historical transaction

database (as shown in figure 4.1). This query checks whether the same cardholder has made another transaction at a physical store in a different country within a time window of 300 seconds, by comparing the timestamps of the two transactions. The result that is, the presence or absence of a geographical conflict is then passed as an additional input to the DMN model in the form of a boolean flag. The DMN engine uses this flag to determine the final risk score: if the flag is true, the risk score is immediately set to 150 or above, resulting in the transaction being declined. In this way, the DMN model retains its role as a pure decision engine, while the database handles the historical context, making the overall solution both scalable and maintainable.

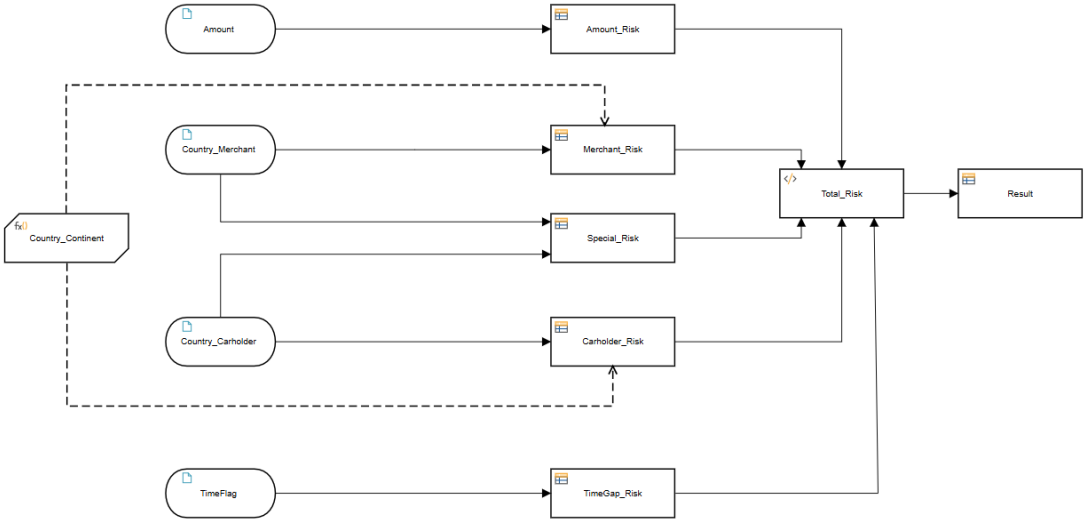

Figure 4.2: The new proposed DMN model

Building on this architecture, a new boolean input variable, called `TimeFlag`, is introduced in the DMN model, whose value is determined externally by the database query described above. If the query returns at least one matching record, the flag is set to true and passed as input to the DMN engine. The decision tables is then extended with an additional rule: if `TimeFlag` is true, the risk score is immediately increased by 150 points, regardless of the other input values, causing the transaction to be declined. The figure 4.2 illustrates the possible updated DMN structure, the corresponding decision table (4.1) with the new rule added and the literal expression.

Rule	**TimeFlag**	**TimeGap_Risk**
1	`True`	150
2	`False`	0

Table 4.1: Decision table of `TimeGap_Risk` (hit policy: U).

4.2 Ontology

The figure 4.2 represents a semantic model designed to support fraud detection in financial transactions by organizing the main entities, attributes, and relationships involved in a transaction monitoring system. The central element of the model is the Transaction entity, which connects a specific Cardholder, Merchant, and Credit Card, allowing financial events to be analyzed together with their contextual information. Each transaction includes attributes such as TransactionID, Timestamp, Amount, RiskScore, and Decision, which describe both operational transaction data and the results of fraud evaluation processes.

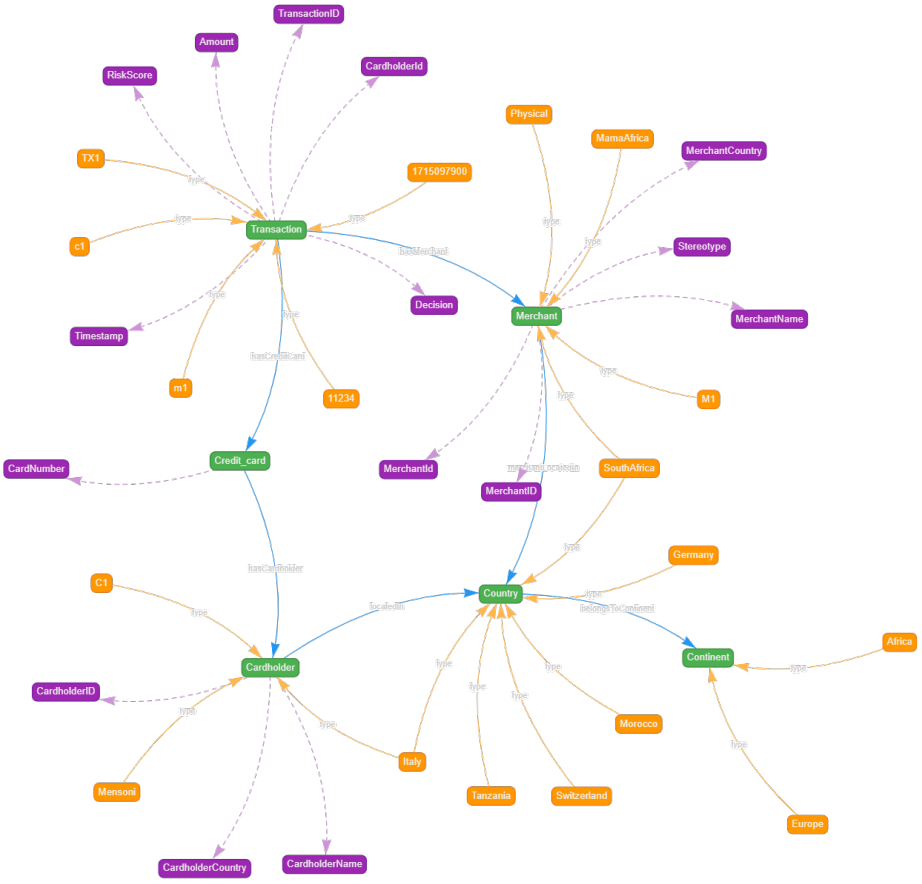

Figure 4.3: Ontology

The Cardholder entity represents the customer performing a transaction and contains descriptive information such as identifiers, names, and geographic location. The introduction of the Credit_card entity extends the model by explicitly representing payment card information through attributes such as CardNumber and by linking the card to

its owner through the hasCardholder relationship. This addition improves the representation of transaction dynamics.
The Merchant entity models the commercial actor receiving the payment and includes information related to merchant identity, category, and geographic location. Geographic knowledge is represented through the classes Country and Continent, connected through hierarchical relationships that capture regional and cross-border transaction patterns. Countries such as Italy, Germany, Switzerland, Morocco, South Africa, and Tanzania are associated with continents including Europe and Africa, enabling contextual reasoning based on geographical risk factors.
The model combines relational and semantic representations by integrating entities, attributes, and graph-based connections such as hasMerchant, hasCreditCard, hasCardholder, locatedIn, and belongsToContinent. This structure helps formalize the relationships used in fraud detection rules while also providing a clear visualization of behavioral and contextual dependencies between transactions, users, merchants, and geographic regions.
Overall, the model provides an extensible and structured representation of the financial transaction domain, supporting both the visualization of the system architecture and the formalization of reasoning processes used for fraud detection and risk assessment.

4.3 Prolog knowledge base implementation

4.3.1 General organization

The Prolog implementation is divided into two main parts: a set of facts, which declare all ground assertions about countries, cardholders, merchants, and transactions; and a set of rules, which encode the reasoning logic used to compute risk scores and classify transactions.

4.3.2 Assertions / Facts

Country predicates

The association between a country and its continent is represented through the predicate `belongs/2`, which takes a country and its corresponding continent as arguments.

```
belongs(albania, europe).
belongs(andorra, europe).
...
belongs(uganda, africa).
belongs(zambia, africa).
belongs(zimbabwe, africa).
```

Transactions and stakeholders

To simulate a database of cardholders, merchants, and transactions, three predicates are defined: `cardholder/3`, `merchant/4`, and `transaction/5`.

```
cardholder(c1, mensoni, italy).
cardholder(c2, meyer, germany).
cardholder(c3, suter, switzerland).
```

```
cardholder(c4, abba, tanzania).
```

The first argument serves as a unique identifier, used to reference the cardholder in other predicates. The remaining arguments represent the cardholder's name and country of residence.

```
merchant(m1, worldofafrica, south_africa, physical).
merchant(m2, windsofthedeserts, morocco, physical).
merchant(m3, mamaafrica, south_africa, physical).
```

The merchant predicate follows the same structure, with the first argument acting as identifier. The fourth argument specifies the type of store (physical or online) which is required by the temporal comparison rule introduced in Task 2.

```
transaction(tx1, c1, m1, 11234, 1715097900).
transaction(tx2, c2, m2, 12, 1715097950).
transaction(tx3, c3, m3, 90, 1715098950).
transaction(tx4, c1, m2, 12, 1715097950).
transaction(tx5, c1, m2, 12, 1715098950).
```

Each transaction is identified by a unique ID and references a cardholder and a merchant by their respective identifiers. The fourth argument is the transaction amount and the fifth is a Unix timestamp, representing the time of the transaction in seconds elapsed since 1 January 1970. Transactions `tx4` and `tx5` are not part of the original assignment scenarios, but were added to test the temporal comparison rule: `tx4` is a second transaction by Mr. Mensoni occurring 50 seconds after `tx1`, while `tx5` occurs 1050 seconds later and therefore falls outside the 5-minute window.

4.3.3 Base rules

This section describes the inference rules defined to solve the task.

An early implementation included a `continent/2` predicate that wrapped `belongs/2` to map countries to continents, mirroring the role of the `Country_Continent` BKM in the DMN model. However, this predicate was later removed, as the risk rules can query `belongs/2` directly without an intermediate mapping step.

The risk-scoring predicates follow a uniform pattern: a set of clauses match specific conditions and return the corresponding score, while a final catch-all clause using the anonymous variable _ returns 0 for any case not explicitly covered.

```
merchant_risk(Country, 10) :-
    Country = italy.
merchant_risk(Country, 20) :-
    Country \= italy,
    belongs(Country, europe), !.
merchant_risk(Country, 60) :-
    belongs(Country, africa), !.
merchant_risk(_,0).
```

```
cardholder_risk(Country, 0):-
    Country = italy.
cardholder_risk(Country, 10):-
    Country \= italy,
    belongs(Country, europe), !.
cardholder_risk(Country, 20) :-
    belongs(Country, africa), !.
cardholder_risk(_,0).


amount_risk(Amount, 10):-
    Amount < 1000,!.
amount_risk(Amount, 60):-
    Amount >= 1000,
    Amount =< 10000,!.
amount_risk(Amount, 90):-
    Amount > 10000,!.


special_risk(Cardholder, Merchant, 30) :-
    Cardholder = switzerland,
    Merchant = south_africa,!.
special_risk(_,_,0).
```

The cut operator `!` is applied in every clause where backtracking could produce multiple results. Once a matching clause is found, the cut prevents Prolog from exploring further alternatives, ensuring that each predicate returns exactly one risk score per invocation. Note that `merchant_risk` and `cardholder_risk` do not require a cut in the first clause, since the explicit equality check `Country = italy` already makes that branch deterministic; the cuts in the subsequent clauses are sufficient to prevent unwanted backtracking into the catch-all.

Transactions comparison

About the constraint proposed in the Task 2, the `score_risk_comparison/2` rule is designed to identify suspicious behavioral patterns related to the temporal proximity of transactions. In particular, the rule verifies whether two different transactions are associated with the same cardholder, are performed in physical stores located in different countries, and occur within a time interval of at most 300 seconds (5 minutes). This condition represents a potentially fraudulent situation, since it would be highly unlikely for the same cardholder to physically perform transactions in different countries within such a short amount of time.

```
score_risk_comparison(T1, T2):-
    transaction(T1, CardholderID, Merchant1, _, Time1),
    transaction(T2, CardholderID, Merchant2, _, Time2),
    merchant(Merchant1, _, Country1, physical),
    merchant(Merchant2, _, Country2, physical),
    T1 \= T2,
```

```
    Country1 \= Country2,
    D is abs(Time1-Time2),
    D =< 300.

time_comparison(T, 150) :-
    transaction(T,_,_,_,_),
    score_risk_comparison(T,_),!.

time_comparison(_,0).
```

The `time_comparison/2` rule builds upon this predicate to incorporate the detected anomaly into the overall fraud evaluation process. When a transaction satisfies the suspicious pattern identified by `score_risk_comparison/2`, the rule assigns an additional risk contribution of 150 points to the transaction. The cut operator (!) is used to ensure that, once a suspicious pattern has been detected, Prolog stops searching for alternative matching solutions and returns only a single risk contribution. If no suspicious temporal pattern is found, the rule assigns a value of 0, meaning that no additional temporal risk is added to the final score.

Summation and detection predicates

At the end, to obtain the overall risk score associated with a transaction, `the score_risk/2` rule is responsible for calculating its. To achieve this, the rule retrieves all the relevant information related to the transaction, including the cardholder, the merchant, the transaction amount, and the involved countries. The total score is then computed by aggregating the different partial risk contributions generated by the previously defined rules: the cardholder risk, the merchant risk, the amount risk, the special geographical risk, and the temporal comparison rule related to suspicious transactions performed within a short time interval. The final score therefore represents a comprehensive evaluation of the transaction risk obtained by combining all these independent factors.

```
score_risk(TransactionID, Score) :-
    transaction(TransactionID, CardholderID, MerchantID, Amount, _ ),
    cardholder(CardholderID, _ , CardholderCountry),
    merchant(MerchantID, _ , MerchantCountry,_),
    cardholder_risk(CardholderCountry, CR),
    merchant_risk(MerchantCountry, MR),
    amount_risk(Amount, AR),
    special_risk(CardholderCountry, MerchantCountry, SR),
    time_comparison(TransactionID, TC),
    Score is CR + MR + AR + SR +TC.
```

Once the total risk score has been calculated, `the detect/3` rule is used to classify the transaction into one of the possible outcomes of the fraud detection system. In particular, a transaction is considered accepted when the score is less than or equal to 100, stopped when the score is between 101 and 150, and rejected when the score exceeds 150. This final rule represents the decision-making component of the model, translating the numerical risk evaluation into a concrete operational decision.

```
detect(TransactionID, accepted, Score):-
```

```
    score_risk(TransactionID, Score),
    Score =< 100.

detect(TransactionID, stopped, Score):-
    score_risk(TransactionID, Score),
    Score > 100,
    Score =< 150.

detect(TransactionID, rejected, Score):-
    score_risk(TransactionID, Score),
    Score > 150.
```

Overall, this structure allows the knowledge base to clearly separate static domain information from the reasoning mechanisms used for fraud detection. By combining declarative facts with logical rules, the system is able to model the transaction domain in a modular and extensible way, while automatically deriving risk evaluations and final decisions through inference. This organization also improves readability and maintainability, making it easier to extend the model with additional rules or new fraud detection patterns in the future.

5. Task 3

5.1 Define RDF schema

A Protégé ontology has been developed in fraud detection systems that defines an RDF schema capable of representing all the elements involved in the fraud detection process. The main classes that make up the scheme are six: `Transaction` for transactions, `Cardholder` for cardholders, `Creditcard` for credit cards, `Merchant` for merchants, `Country` for countries, and `Continent` for continents.
These classes are connected to each other via object properties that express the meaningful relationships of the domain. Specifically, a transaction is associated with a card via `hasCreditcard` and an merchant via `hasMerchant`; the card is in turn linked to its holder via `hasCardholder`; the cardholder resides in a country via `locatedIn`, while the merchant is located in a country via `merchantLocatedIn`; finally, each country belongs to a continent via `belongsToContinent`. To store concrete data for transactions and other items, several data properties have been defined: `Amount` and `Timestamp` record the amount and time of each transaction, respectively; `StoreType` indicates whether the merchant operates in a physical store or online; and various properties such as `CardholderName`, `MerchantName`, `CountryName`, and `ContinentName` retain descriptive names. The schema also includes properties designed to accommodate the intermediate and final results of the risk calculation, namely`MerchantRisk`, `CardholderRisk`, `AmountRisk`, `SpecialRisk`, `TimeRisk`, `RiskScore`, and `decision`. All properties are declared with their respective domains and ranges, ensuring that each value is placed in the correct class. This scheme therefore provides a formal, complete and reusable basis for representing all the information needed by the fraud detection system.

5.2 Define test case

Based on the defined RDF schema, all the necessary individuals were added to the ontology to represent the three scenarios provided in the project description, along with some additional cases to verify the correct functioning of the rules. Regarding cardholders, the individuals `mensoni` residing in `italy`, `meyer` residing in `germany`, and `suter` residing in `switzerland` were created. A credit card was associated with each of them, namely `card1`, `card2`, and `card3` respectively, using the `hasCardholder` property. For merchants, `worldOfAfrica` located in `south_africa`, `windsOfTheDesert` located in `morocco`, `mamaAfrica` also located in `south_africa`, and an additional Italian merchant called `mammaMia` were defined to test queries related to Italian merchants. All merchants were declared as physical stores through the `StoreType` property.

Six transactions were inserted. Transaction `tx1` represents Mensoni purchasing an item worth 11,234 at the `worldOfAfrica` store in South Africa. Transaction `tx2` represents Meyer paying 12 for a meal at the `windsOfTheDesert` restaurant in Morocco. Transaction `tx3` describes Suter having a 90 at the `mamaAfrica` restaurant in South Africa. To test the five-minute temporal rule, two additional transactions were added using Mensoni's same card: `tx4` occurs 50 seconds after `tx1` at the `windsOfTheDesert` restaurant in Morocco, while `tx5` occurs 1,050 seconds later, thus exceeding the five-minute threshold. Finally, `tx6` represents a 512 transaction by Meyer at the Italian merchant `mammaMia`, useful for verifying queries related to Italian merchants. Each transaction was enriched with `Amount` and `Timestamp` values, the latter expressed in UNIX timestamp format to allow temporal difference calculations. For some transactions, intermediate risk values and the final decision were also precomputed and inserted, as shown by the `AmountRisk`, `CardholderRisk`, `MerchantRisk`, `SpecialRisk`, `TimeRisk`, `RiskScore`, and `decision` properties present on each individual.
This set of test cases covers all the scenarios expected by the risk rules and allows the system's behaviour to be fully validated.

5.3 SWRL rule

To translate the risk heuristics defined in the previous tasks into formally executable rules, several SWRL rules have been implemented in the ontology. There are three rules for merchant risk. Rule `MerchantRisk1` assigns 60 points if the merchant is located in Africa. Rule `MerchantRisk2` assigns 10 points if the merchant is located in Italy. Rule `MerchantRisk3` assigns 20 points if the merchant is located in Europe but not in Italy. Similarly, three rules have been defined for cardholder risk. Rule `CardholderRisk1` assigns 0 points if the cardholder resides in Italy. Rule `CardholderRisk2` assigns 10 points if the cardholder resides in a European country other than Italy. Rule `CardholderRisk3` assigns 20 points if the cardholder resides in Africa. Regarding the transaction amount, the three rules are `AmountRisk1`, which assigns 10 points for amounts below 1000 euros, `AmountRisk2`, which assigns 60 points for amounts between 1000 and 10000 euros, and `AmountRisk3`, which assigns 90 points for amounts above 10000 euros. Special risk is handled by rule `SpecialRisk1`, which assigns 30 points when the cardholder is Swiss and the merchant is located in South Africa, while rule `SpecialRisk2` assigns 0 points in all other cases. The temporal rule, representing the novelty introduced in Task 2, is implemented by `TimeRisk`, which identifies pairs of transactions sharing the same credit card, involving merchants in different countries and of physical type, with a time difference not exceeding 300 seconds, and assigns 150 points in such a case. Rule `TimeRisk2` assigns 0 points when this condition is not met. Once all partial contributions have been calculated, the `SumRule` sums them using the built-in `add` operator and produces the overall `RiskScore` for each transaction. At this point, three rules determine the final decision: `AcceptedRule` assigns "accepted" if the score is less than or equal to 100, `BlockRule` assigns "stopped" if the score is between 101 and 149, and `RejectRule` assigns "rejected" if the score is greater than or equal to 150.

5.4 SPARQL query

The first SPARQL query simply retrieves the complete list of all transactions in the knowledge graph. Does the query select the variable `?tx` and applies a pattern that requires that `?tx` is of type `Transaction`. This query is useful for getting an overview of the data loaded into the ontology and for verifying that all individuals were imported correctly.

```
#1 all transactions

SELECT ?t
WHERE {
  ?t rdf:type task_3:Transaction .
}
```

The second query extracts transactions whose merchant is based in South Africa. To achieve this, the query uses two patterns: does the first bind the transaction `?tx` to his merchant `?m` through the `hasMerchant` property, while the second requires that the merchant `?m` is located in South Africa via the property `merchantLocatedIn` which points to the individual `south_africa`. This query makes it possible to quickly identify all transactions involving South African traders, which, according to the risk rules, result in an increase of 60 points.

```
#2 transactions with merchants from South

Africa
SELECT ?t ?m
WHERE {
  ?t rdf:type task_3:Transaction ;
     task_3:hasMerchant ?m .
  ?m task_3:merchantLocatedIn task_3:south_africa .
}
```

The third query retrieves transactions whose cardholders reside in a European country. Graph navigation occurs through a chain of properties: from the transaction `?tx` do we go back to the credit card `?card` via `hasCreditcard`, from the card you get the cardholder `?ch` with `hasCardholder`, from the cardholder you access the country of residence `?c` with `locatedIn`, and finally it is verified that this country belongs to the European continent by `belongsToContinent` which points to the individual `europe`. This query is particularly useful for analyzing the behavior of European cardholders, who contribute to risk differently depending on whether they reside in Italy or other European countries.

```
#3 Transactions with cardholders from Europe

SELECT ?t ?ch
WHERE {
  ?t rdf:type task_3:Transaction ;
     task_3:hasCreditcard ?card .
  ?card task_3:hasCardholder ?ch .
```

```
  ?ch task_3:locatedIn ?c .
  ?c task_3:belongsToContinent task_3:europe .
}
```

The fourth query extracts transactions whose merchants are based in Italy, showing for each of them the value of the risk associated with the merchant. Does the query bind the transaction `?tx` to his merchant `?m` via `hasMerchant`, verifies that the merchant is located in Italy with `merchantLocatedIn` pointing to `italy`, and retrieves the merchant's risk value through the `MerchantRisk` data property associated with the transaction.

```
#4 Transactions and their merchant risk for transactions with merchant from It

SELECT ?t ?mr
WHERE {
  ?t rdf:type task_3:Transaction ;
     task_3:hasMerchant ?m ;
     task_3:MerchantRisk ?mr .
  ?m task_3:merchantLocatedIn task_3:italy .
}
```

5.5 Special SPARQL query

After activating the reasoner that applies all SWRL rules, the final query retrieves transactions considered fraudulent based on the risk score. The query selects the transaction `?t` and its `RiskScore` ?score, requiring that the transaction is of type `Transaction` and that the risk score is greater than 100. This approach is more direct than filtering on the decision, since the threshold of 100 points represents the limit beyond which a transaction is no longer automatically accepted. Thus, both stopped transactions (with a score between 101 and 150) and rejected transactions (with a score above 150) are selected.

```
#5 Retrieve the fraudulent transactions.

SELECT ?t ?score
WHERE {
  ?t rdf:type task_3:Transaction ;
     task_3:RiskScore ?score .
  FILTER(?score > 100)
}
```